

Step-by-Step Python Data Cleaning Process - HR Analytics Project

This document explains the complete step-by-step Python data cleaning process performed on the messy HR dataset before loading it into PostgreSQL and Power BI.

Step 1: Import Required Libraries

```
import pandas as pd
from sqlalchemy import create_engine
import matplotlib.pyplot as plt
```

Explanation: Imported libraries for data analysis, database connection, and visualization.

Step 2: Load Dataset

```
df = pd.read_csv("messy_hr_dataset_100k.csv")
```

Explanation: Loaded the HR dataset into a Pandas DataFrame.

Step 3: View Dataset Information

```
print(df.head())
```

	Employee_ID	Age	Gender	Department	Job_Role	Monthly_Income	Years_At_Company	Work_Life_Balance	Job_Satisfaction	Overtime	Performance_Rating	Attrition
0	7864	34	Male	Admin	Business Analyst	119300.0	8	3	4.0	No	3.0	Yes
1	92230	36	Female	Marketing	Accountant	NaN	26	3	1.0	Yes	3.0	No
2	99080	48	Male	IT	Accountant	176808.0	19	1	3.0	Yes	4.0	No
3	88172	54	male	Marketing	HR Executive	77497.0	13	3	3.0	No	1.0	No
4	85183	41	male	Finance	Sales Executive	112780.0	15	3	1.0	Yes	3.0	No

Process finished with exit code 0

```
print(df.shape)
```

(100500, 12)
Process finished with exit code 0

```
print(df.info())
```

```
<class 'pandas.DataFrame'>
RangeIndex: 100500 entries, 0 to 100499
Data columns (total 12 columns):
 #   Column                Non-Null Count  Dtype  
---  -
 0   Employee_ID           100500 non-null int64  
 1   Age                   100500 non-null int64  
 2   Gender                100500 non-null str  
 3   Department            100500 non-null str  
 4   Job_Role              100500 non-null str  
 5   Monthly_Income        98495 non-null float64 
 6   Years_At_Company      100500 non-null int64  
 7   Work_Life_Balance     100500 non-null int64  
 8   Job_Satisfaction      98493 non-null float64 
 9   Overtime              100500 non-null str  
10   Performance_Rating    100200 non-null float64 
11   Attrition             100500 non-null str  
dtypes: float64(3), int64(4), str(5)
memory usage: 9.2 MB
None

Process finished with exit code 0
```

Explanation: Checked dataset structure, number of rows and columns, and data types.

Step 4: Check Missing Values

```
print(df.isnull().sum())
```

```
Employee_ID      0
Age              0
Gender           0
Department       0
Job_Role         0
Monthly_Income    2005
Years_At_Company  0
Work_Life_Balance 0
Job_Satisfaction  2007
Overtime         0
Performance_Rating 300
Attrition        0
dtype: int64
```

Explanation: Identified columns containing null values.

Step 5: Fill Missing Monthly Income Values

```
df['Monthly_Income'] = df['Monthly_Income'].fillna(df['Monthly_Income'].median())  
print(df['Monthly_Income'])
```

Explanation: Used median to fill missing salary values because salary data may contain outliers.

Step 6: Fill Missing Job Satisfaction Values

```
df['Job_Satisfaction'] = df['Job_Satisfaction'].fillna(df['Job_Satisfaction'].median())  
print(df['Job_Satisfaction'])
```

Explanation: Filled missing job satisfaction values using median.

Step 7: Fill Missing Performance Rating Values

```
df['Performance_Rating'] = df['Performance_Rating'].fillna(df['Performance_Rating'].mode()[0])  
print(df['Performance_Rating'])
```

Explanation: Used mode because performance rating is categorical/discrete data.

Step 8: Check Duplicate Records

```
print(df.duplicated().sum())
```

Explanation: Identified duplicate employee records.

Step 9: Remove Duplicate Records

```
df = df.drop_duplicates()  
print(df)  
print(df.duplicated().sum())
```

Explanation: Removed duplicate rows to improve data accuracy.

Step 10: Fix Department Name Inconsistencies

```
df['Department'] = df['Department'].replace({  
    'Human Resources': 'HR',  
    'human resources': 'HR',  
    'HR': 'HR',  
    'Human Resources': 'HR',  
    'Humann Resources' : 'HR'  
})  
print(df['Department'])
```

Explanation: Standardized department names for consistent analysis.

Step 11: Fix Gender Inconsistencies

```
df['Gender'] = df['Gender'].replace({  
    'M' : 'Male',  
    'male' : 'Male',  
    'F' : 'Female',  
    'female' : 'Female'  
})  
print(df['Gender'])
```

Explanation: Standardized gender values into a single format.

Step 12: Remove Extra Spaces

```
df['Department'] = df['Department'].str.strip()  
print(df['Department'])
```

Explanation: Removed unwanted leading and trailing spaces from department names.

Step 13: Find Negative Salary Values

```
print(df[df['Monthly_Income'] < 0])
```

Explanation: Detected invalid negative salary values.

Step 14: Convert Negative Salaries to Positive

```
df['Monthly_Income'] = df['Monthly_Income'].abs()  
print(df['Monthly_Income'])
```

Explanation: Converted negative salary values into positive values.

Step 15: Check Data Types

```
print(df.dtypes)
```

Explanation: Verified column data types after cleaning.

Step 18: Rename Column Names

```
df.columns = df.columns.str.lower()  
print(df.columns)
```

Explanation: Converted column names to lowercase for SQL compatibility.

Step 19: Final Null Value Check

```
print(df.isnull().sum())  
print(df.shape)  
print(df['department'].unique())
```

Explanation: Ensured there were no remaining missing values.

Step 20: Save Cleaned Dataset

```
Print(df.to_csv("Cleaned_hr_Dataset.csv", index=False))
```

Explanation: Saved the cleaned dataset into a new CSV file.

Step 21: Connect Python to PostgreSQL

```
username = "postgres"  
password = "1234"  
host = "localhost"  
port = "5432"  
database = "HR_Analysis_&_Employee_Attrition"
```

```
engine = create_engine(f"postgresql+psycopg2://{username}:{password}@{host}:{port}/{database}")
```

Explanation: Created a PostgreSQL database connection using SQLAlchemy.

Step 22: Load Cleaned Data into PostgreSQL

```
table_name = "hr_data"  
df.to_sql(table_name,engine,if_exists="replace",index=False)  
print(f"Data successfully loaded into table '{table_name}' in database '{database}'.")
```

Explanation: Loaded the cleaned HR dataset into PostgreSQL for SQL analysis.

Conclusion

The HR dataset was successfully cleaned using Python and Pandas. The cleaned dataset was then loaded into PostgreSQL for SQL analysis and later used in Power BI for dashboard creation and business insights.